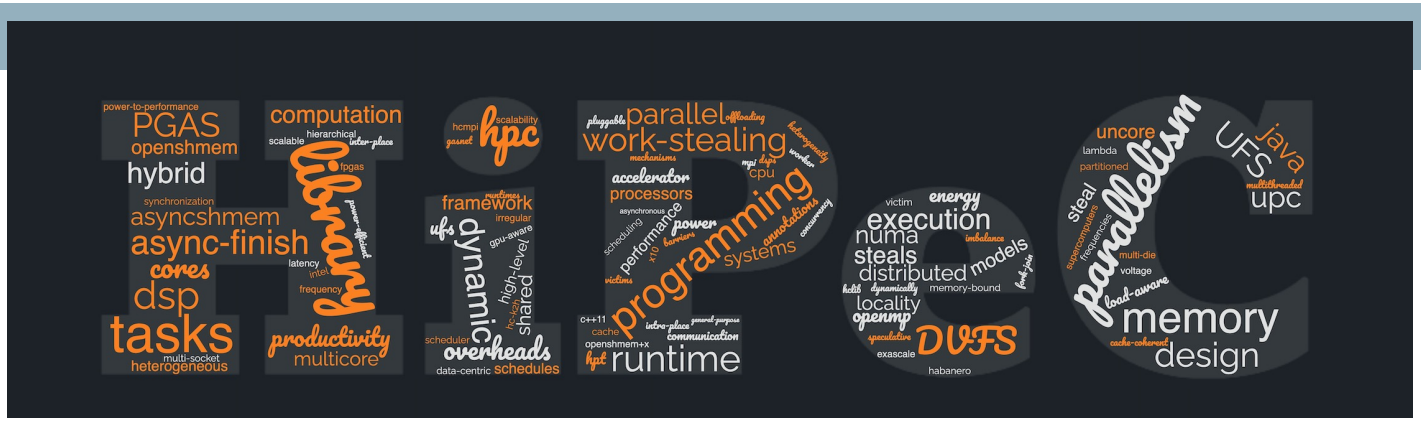
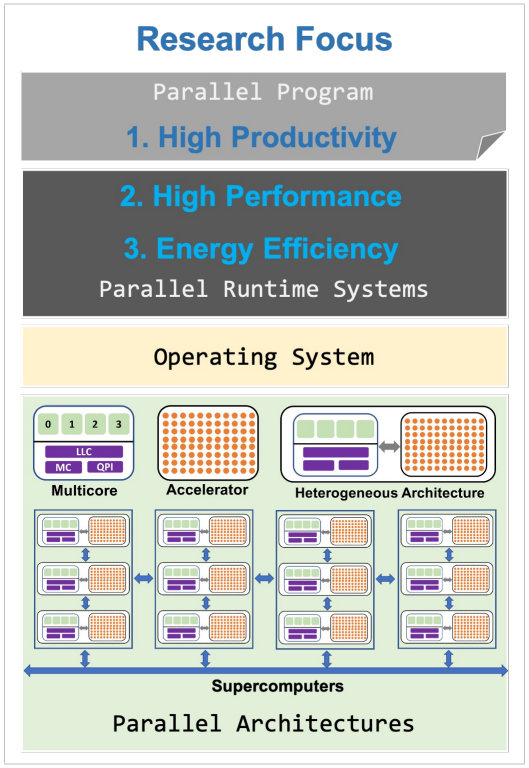
Lecture 01: Course Introduction

Vivek Kumar
Computer Science and Engineering
IIIT Delhi
vivekk@iiitd.ac.in

About me

- https://hipec.github.io/





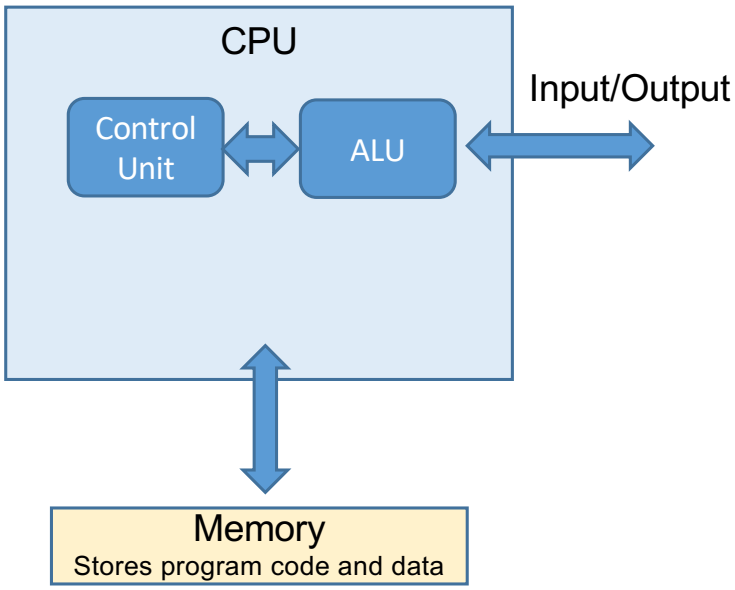
I spend my time at the HiPeC Lab or roaming in the majestic Himalayas..

Today's Lecture

- Modern computing architecture
- High-level overview of operating systems
- Roles of an OS
- Challenges in modern OS
- Course evaluation and logistics

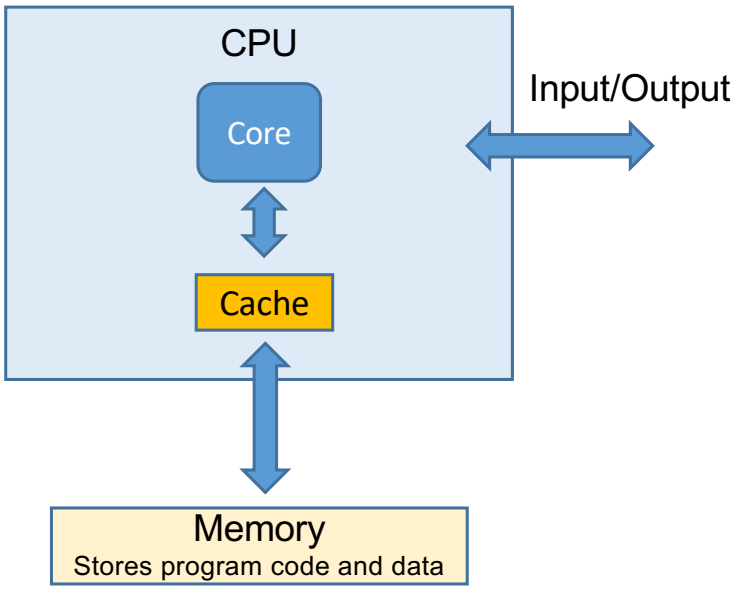
Let us first quickly try to understand the modern computer architecture

Von Neumann Architecture & Associated Issues (1/4)



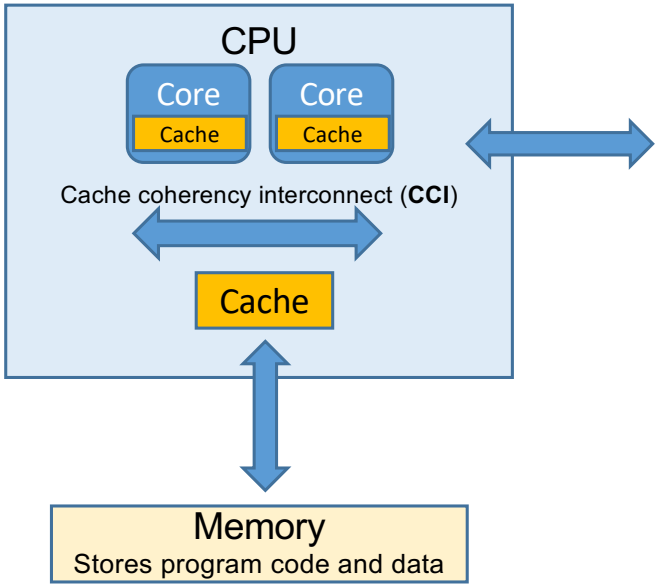
- John Von Neuman in 1945 came up with the architecture for computers that we even use today (albeit with several changes)
  - Prior to that, Harvard architecture used to exist
    - Separate storage for instructions and data
- Next Problem?
  - Memory bottleneck
    - Access latency to memory quite high
      - High CPU stalls while fetching code and data

Von Neumann Architecture & Associated Issues (2/4)



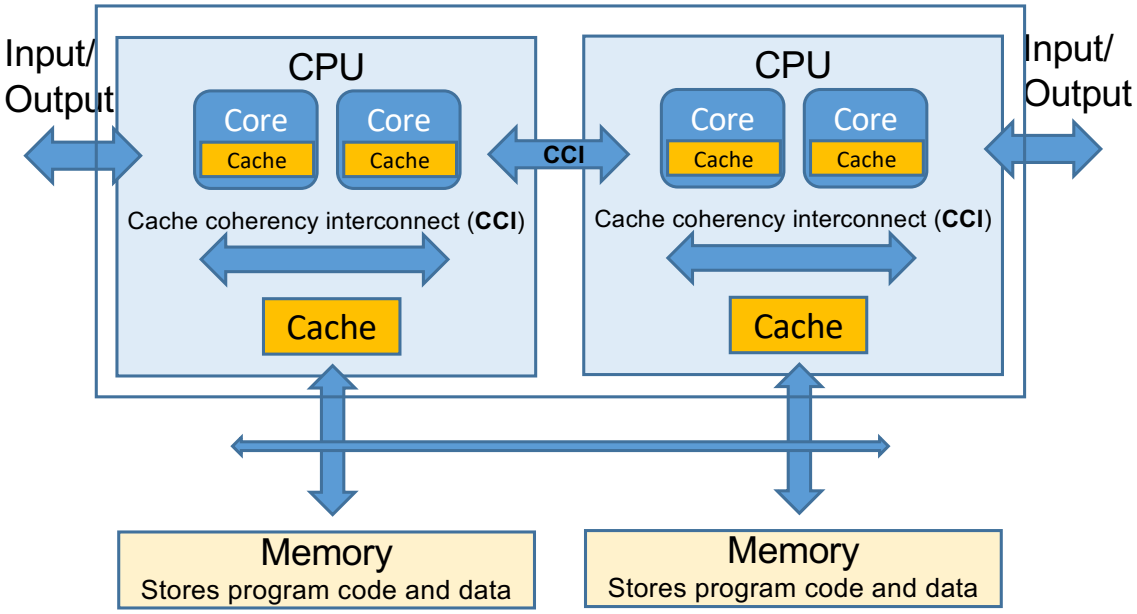
- Solution
  - Add cache on the CPU chip to store frequently accessed memory
- Next Problem?
  - Performance bottleneck
    - Design issues with increasing the performance of single core processor
      - High heat dissipation
        - Even capable of melting the processor!
      - High power consumption

Von Neumann Architecture & Associated Issues (3/4)



- Solution (around 2004)
  - Add more cores to achieve better performance instead of increasing the performance of a single core
    - Still maintains the Moore's law
  - Add cache coherency interconnect (CCI) to fetch data on one core from the other core's cache instead of going all the way up to main memory
- Next problem
  - Performance bottleneck
  - Modern applications requires more **performance** and **data**
    - How to improve the performance and data storage of a single system?

Von Neumann Architecture & Associated Issues (4/4)

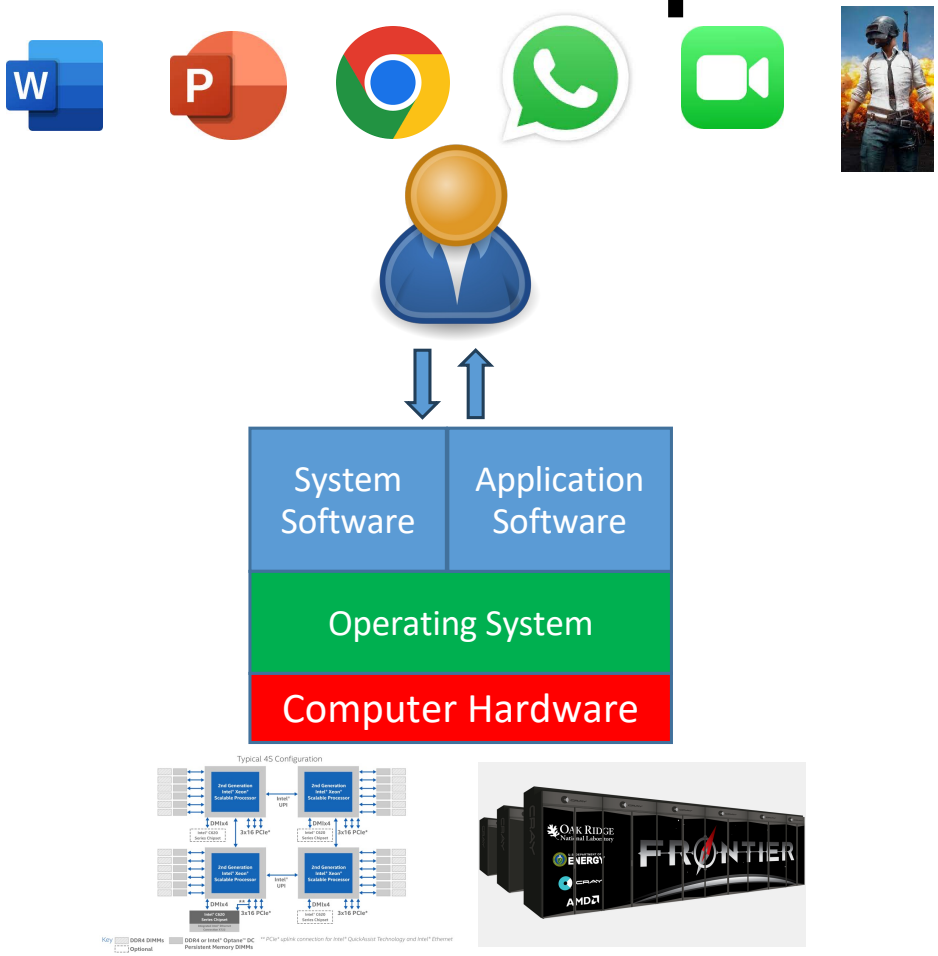


- Solution
  - Add more processors (a.k.a. sockets) on the same motherboard
  - Provide local memory banks at each socket
    - Each CPU can access local and remote memory banks

Today's Lecture

- Modern computing architecture
- High-level overview of operating systems
- Roles of an OS
- Challenges in modern OS
- Course evaluation and logistics

What is an Operating System?



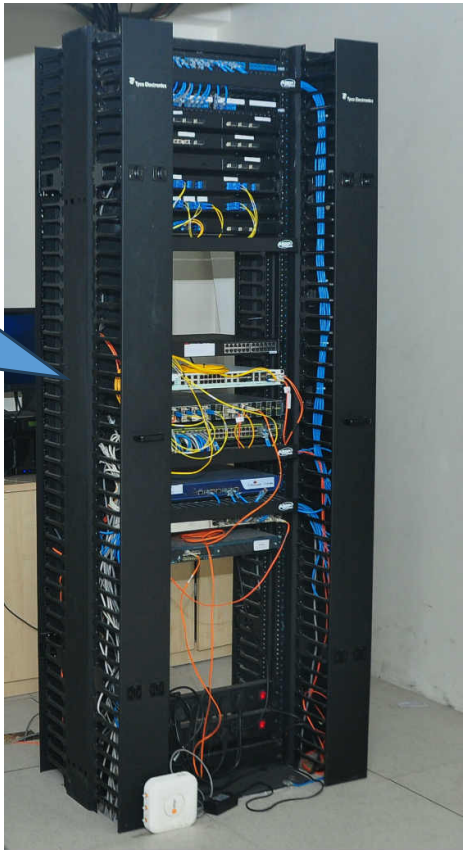
- OS is a piece of software whose job is the manage the computer's resources for its users and applications

Analogy: The Hardware

Analogy: Disk & DRAM

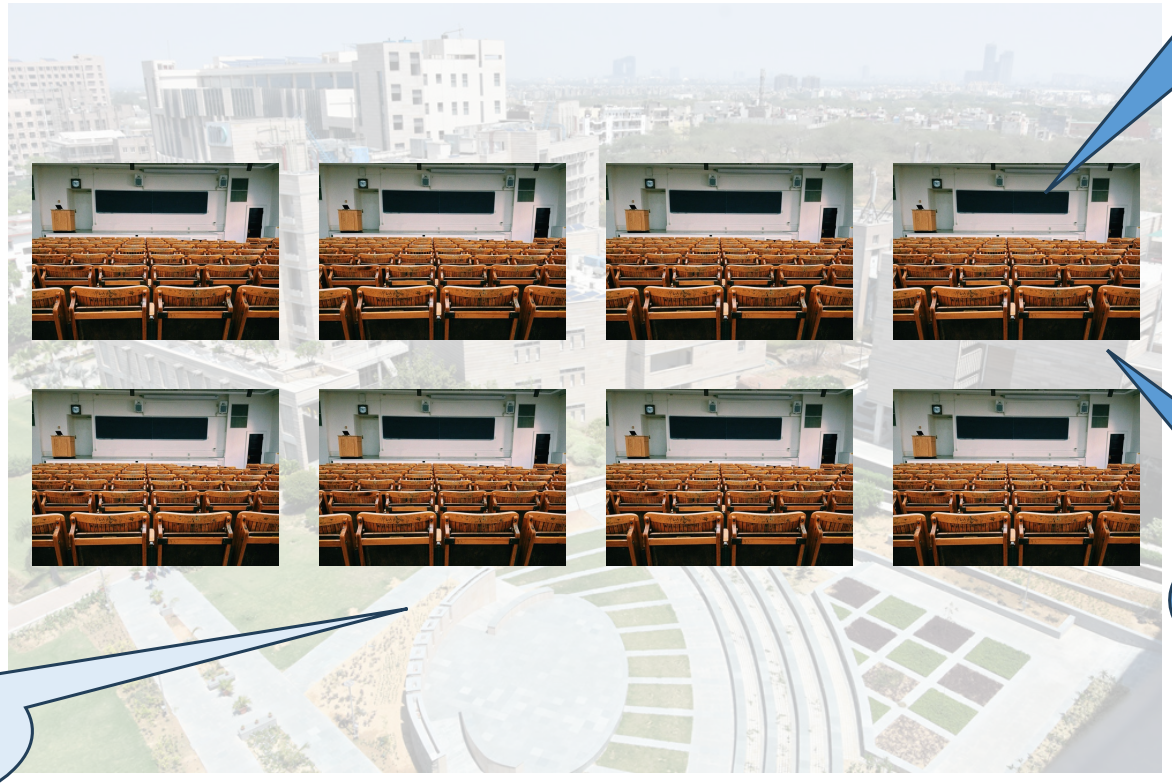
Library – persistent storage of data (Disk)

Data is read from the Disk (**Library**) and stored in central server (**DRAM**) in the form of **lecture slides**



Any familiarity with memory hierarchy?

Analogy: The Multicore CPU



Board (Cache)

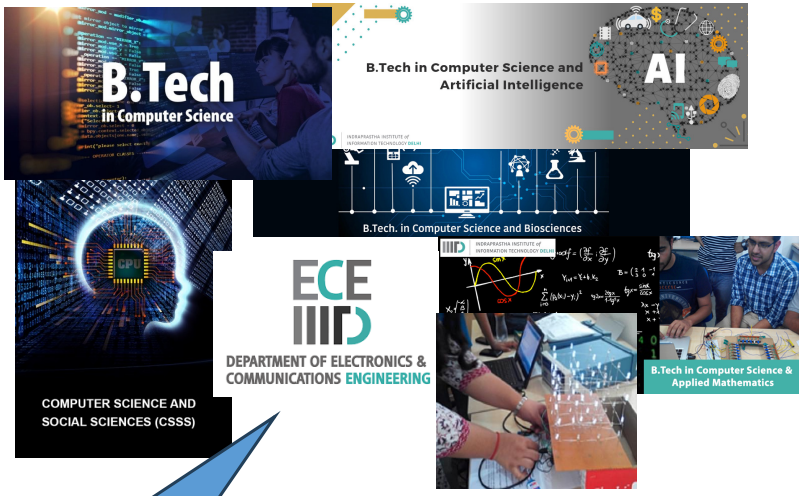
Classroom (Core)

Fixed number of Cores

Analogy: The Complete Hardware

Analogy: Operating System & Applications

OS: Faculties and Staff



Applications: Several BTech/MTech programs

Each **application** has several **Tasks** to complete (courses)

Major Goals of Operating Systems

1. Virtualization
   - CPU
   - Memory
2. Concurrency
3. Persistence
4. Protection and security

Analogy: Virtualization (CPU cores)

Scheduling: Time table is prepared for each semester for smooth running of each program

- The operating system gives an "**illusion**" to each application that the entire hardware is only for that application
  - So many applications can execute on the same hardware with limited resources (fixed number of cores, DRAM, and disk)

Analogy: Virtualization (CPU cores)

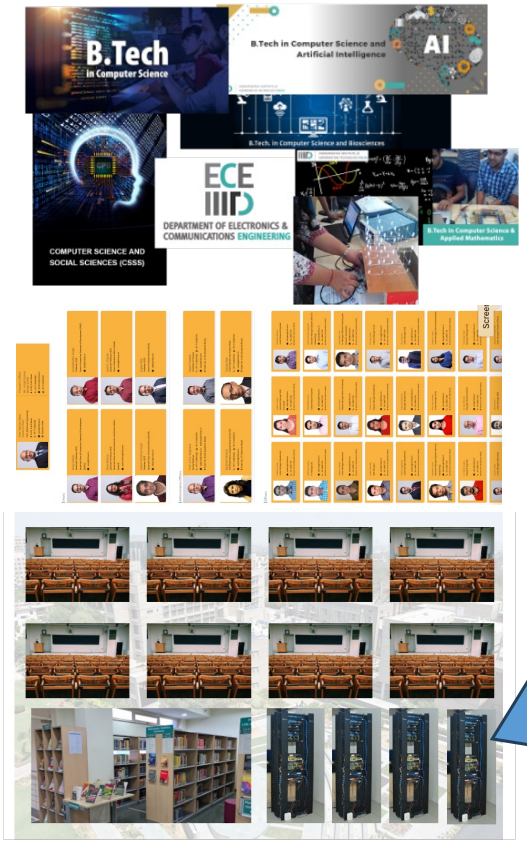
Every program has many courses, each with several lectures

- Application stored in file v/s running applications
- Each running application (**Process**) has some code and data associated with it (e.g., course and lecture materials)
  - Faculty prepares the lecture by fetching materials from library (**Disk**) and then preparing lecture material/slides saved on servers (**DRAM**)
  - Lectures are **Scheduled** in a classroom where the faculty uses the whiteboard (**Cache**) to teach the topic

Analogy: Virtualization (CPU cores)

- Only one **Task** can run on a CPU core at any given time
  1. A classroom is used to run lectures from different courses
  2. During the lecture, students write down notes from the board (**cache**) onto their notebook (persistent storage)
  3. Next faculty enters the classroom for delivering a different lecture
  4. First, a recap of the topics discussed in the last lecture is carried out before starting the lecture
     - Reloading the context
- Above steps are a very high-level overview of **context switch between processes for time sharing of a CPU core**!

Analogy: Virtualization (Memory)



There are several servers (DRAMs) to store the data associated with each program

- Faculties and admin (OS) stores the data from each program (courses, lectures, documents, etc.) in the servers (DRAMs)
- It gives an illusion that there are dedicated memory spaces for each program
  - In the reality each program's data could be stored in pieces over several DRAMs (physical address)
  - However, this complexity is abstracted away by the O.S.

Analogy: Concurrency

Scheduling algorithm of IIIT-Delhi

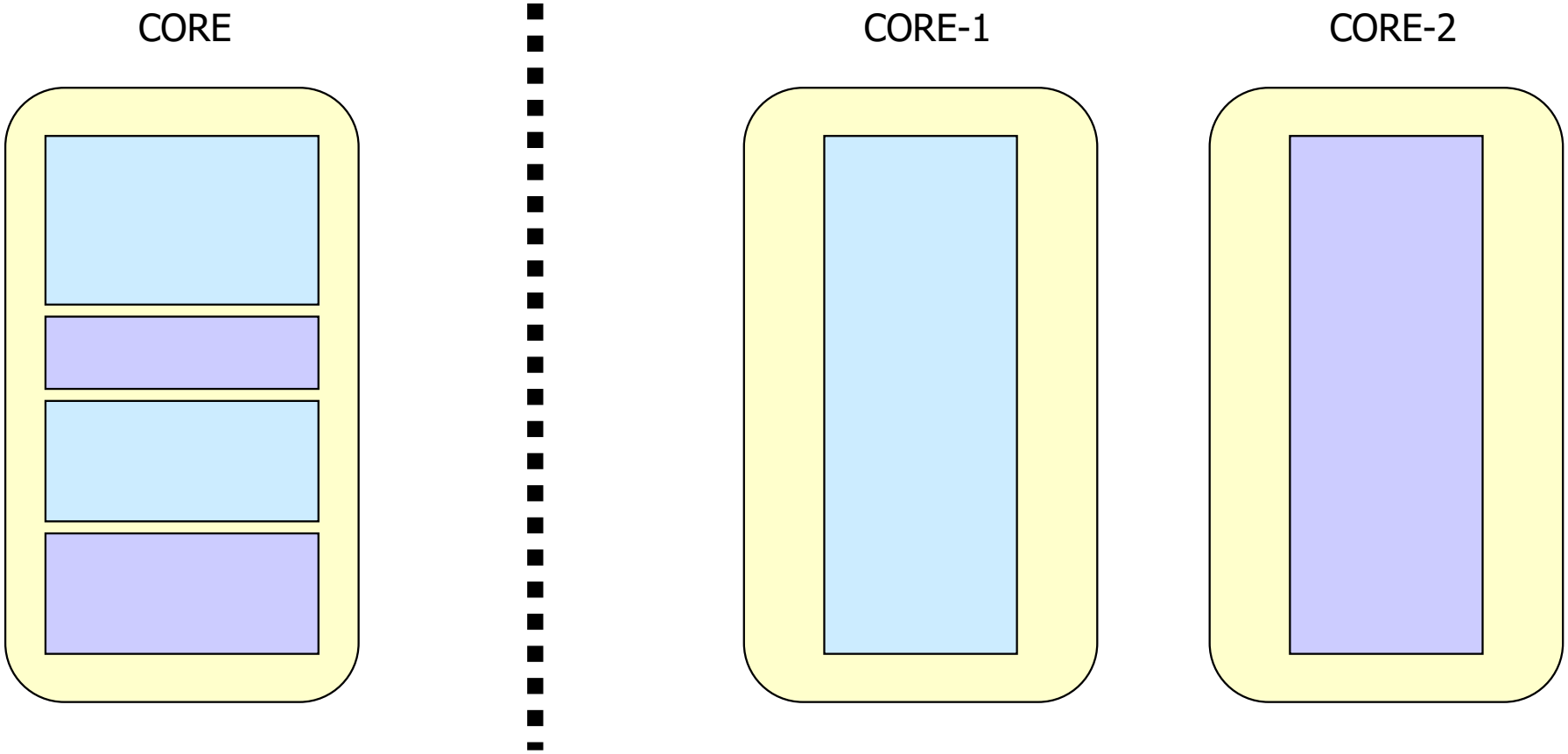
B.Tech. 3rd & 4th Year, M.Tech. 1st & 2nd year, Ph.D. - Monsoon Semester AY 2022-23

Day	8.30-9.00	9-9:30	9:30-10	10-10:30	10:30-11	11-11:30	11:30-12	12-12:30	12:30-1	1-1:30	1:30-2	2-2:30	2:30-3	3-3:30
Mon			CN (Sec A) CN (Sec B) BML WSI IDUDA NDM MLBA CMSMR MAD RA-II	C21 C102 C03 C13 C208 C22 A006 C209 C214 C215	Slot 4	AC DMG QMD ETB ME IAGA DFML BioP ADL	C03 C21 C216 C22 A007 C208 C209 C214 A006	Slot 2						DIP
Tue			CoMeG ML (A) ML (B) LST QM TML AERM DAVP WN NDMA	C03 C11 C21 C13 C22 C208 C215 C24 C213 C210	Slot 1	FOE CGAS IIA SDOS DSc CMOS SC FF ADC WARDI G&M	C02 A006 C213 A106 C210 C211 A007 C21 C12 C208 C209	Slot 5			CGS			ITFW PRP CG
			CN (Sec A) CN (Sec B)	C21 C102	Slot 4	CA Chi	A006 C13	Slot 9						DIP GA

B.Tech. CSE program has several courses, some of which could be scheduled together in the same slot, but in different **classrooms**!

- A program could have several independent tasks (set of instructions) that could executed simultaneously over multicore CPUs
  - Multiple threads of execution
- However, some course has prerequisite courses that the students should complete first
  - Dependencies in multi-threaded execution

Concurrency vs. Parallelism



Analogy: Persistence

- At the end of each semester, academic department collect and permanently save the lecture slides, quiz sheets, assignments and project, and student's answer sheets for mid/end semester exams (for each courses)

Analogy: Protection (Internal) & Security (External)

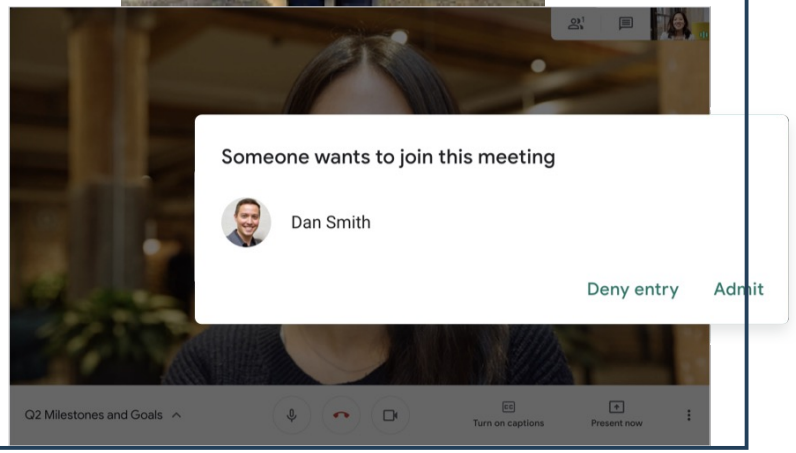
≠

ECE

ARTMENT OF ELECTRONICS &
COMMUNICATIONS ENGINEERING

Course Name	Course Acronym	Course Code
Computer Architecture	CA	CSE511/ECE511

Each program can only access its own data (courses) unless there is some authorization obtained by the O.S



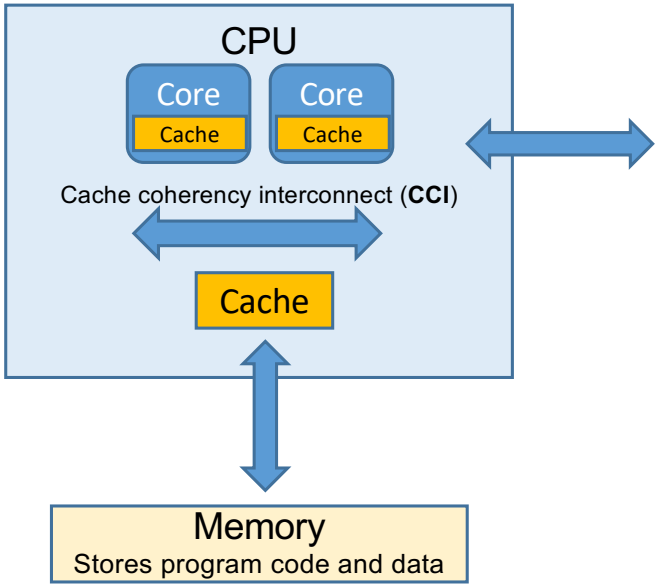
Today's Lecture

- Modern computing architecture
- High-level overview of operating systems
- Roles of an OS
- Challenges in modern OS
- Course evaluation and logistics

Roles of an Operating System

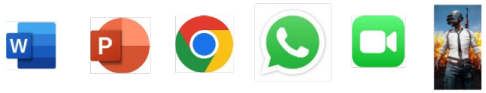
- Referee
  - Manages all the shared resources in the computer
- Illusionist
  - Each application thinks that the entire computer belong to itself
- Glue
  - Offers standard services to simplify application development and facilitate sharing

Roles of an Operating System (Example-1)



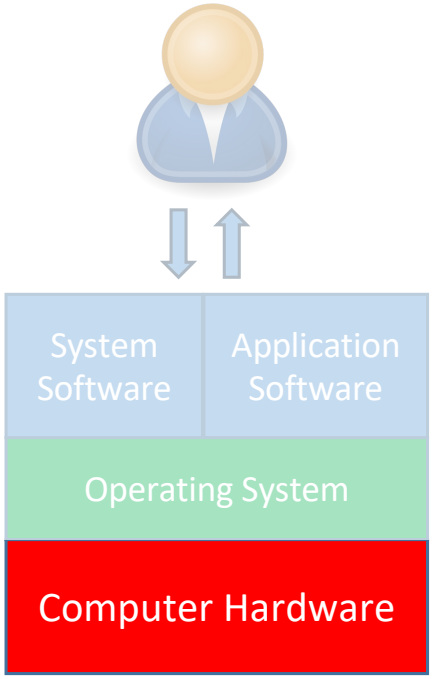
- Let us try to understand it from the perspective of a desktop/laptop

Roles of an Operating System



- **Referee**
  - Each application will be launching several tasks
    - Gaming app will require graphic rendering, network play support, taking player input via keyboard/mouse, etc.
    - Zoom will require tasks such as network connection, access to mike/camera, graphic rendering, etc.
  - As there are several cores, each task can run on a different core
    - But, there are only finite number of resources (cores, caches, input/output devices, memory etc.)
- **Illusionist**
  - Allows the application to have hundreds of tasks (several times more than available cores), infinite amount of memory (DRAMs are only in few GBs), etc.
- **Glue**
  - Standard APIs for network connection, accessing input/output devices, etc.

Operating System Specific to Hardware

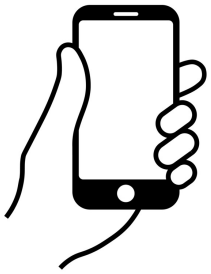


- Throughput oriented (manages several tasks)
- E.g., Windows, Linux, MacOS

Laptop/Desktop Operating System

- Lightweight
- Highly user friendly (GUI)
- Smaller memory footprint (more than RTOS)
- E.g., Andriod, iOS

Mobile Operating System



- Time bound and limited tasks
- May work with small memory
- E.g., Nucleus RTOS

Real time Operating System

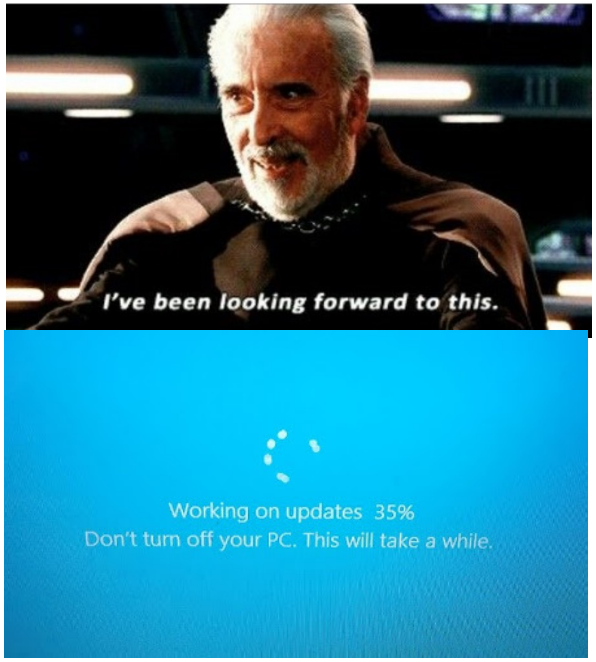
Today's Lecture

- Modern computing architecture
- High-level overview of operating systems
- Roles of an OS
- Challenges in modern OS
- Course evaluation and logistics

Major Challenges in Modern OS

- Dependability

Me: *restarts computer because it's lagging*
Windows Update:



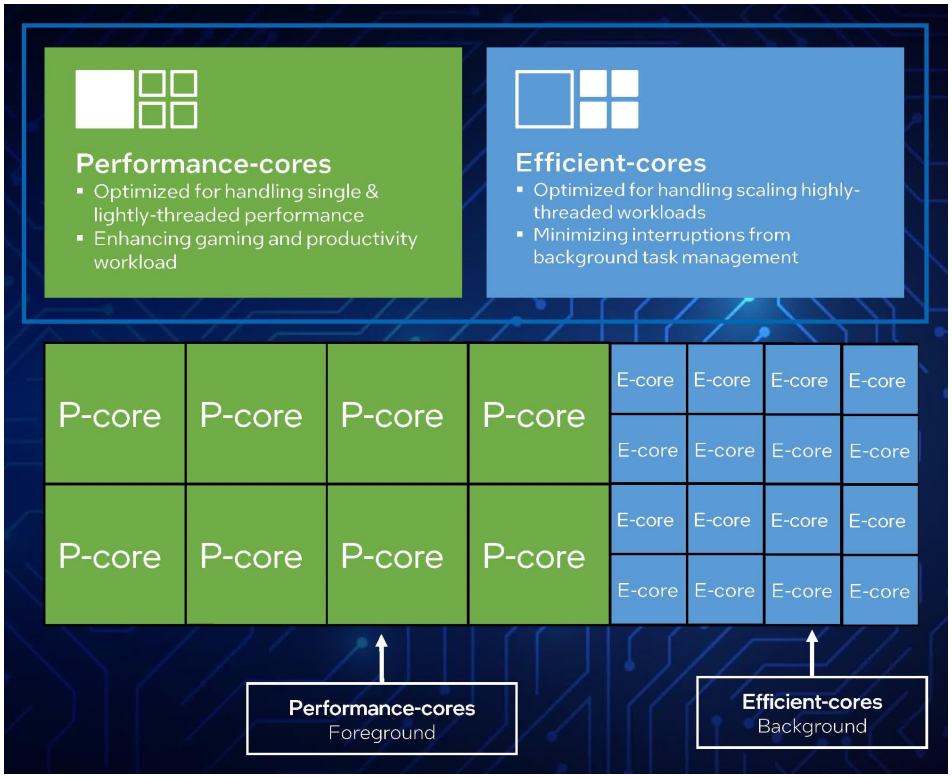
Predictable? Reliable?
(frequent bug fixes, updates…)

Predictable?
Reliable?

V/S

Major Challenges in Modern OS

- Performance and Energy



Which tasks should run on P-core and which ones on E-cores?

What should be the processor core frequency while running a particular task? High frequency can give better performance, but requires more energy, and vice-versa

Images source: https://www.techspot.com/news/94919-preview-core-i9-13900-engineering-sample-performance-looks.html

Major Challenges in Modern OS

- Protection and Security

They are lacking **Security**

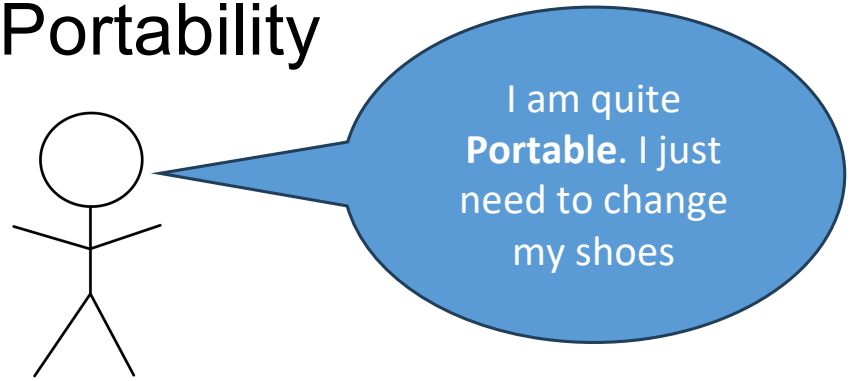
Let's unleash our arsenal

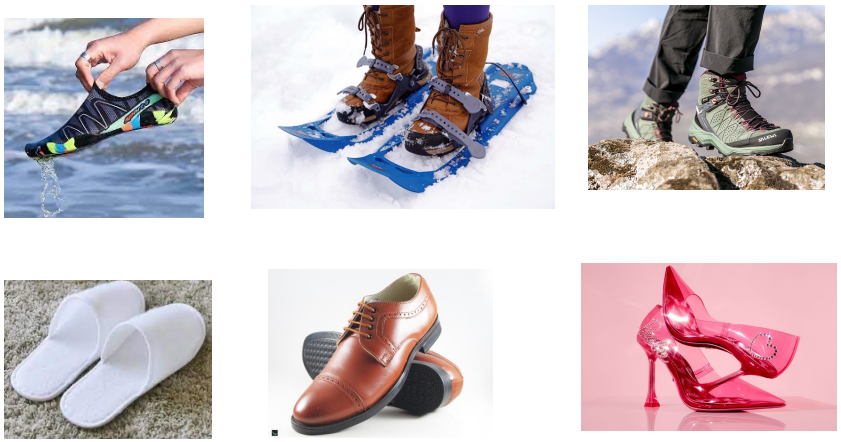
We are **Protected**

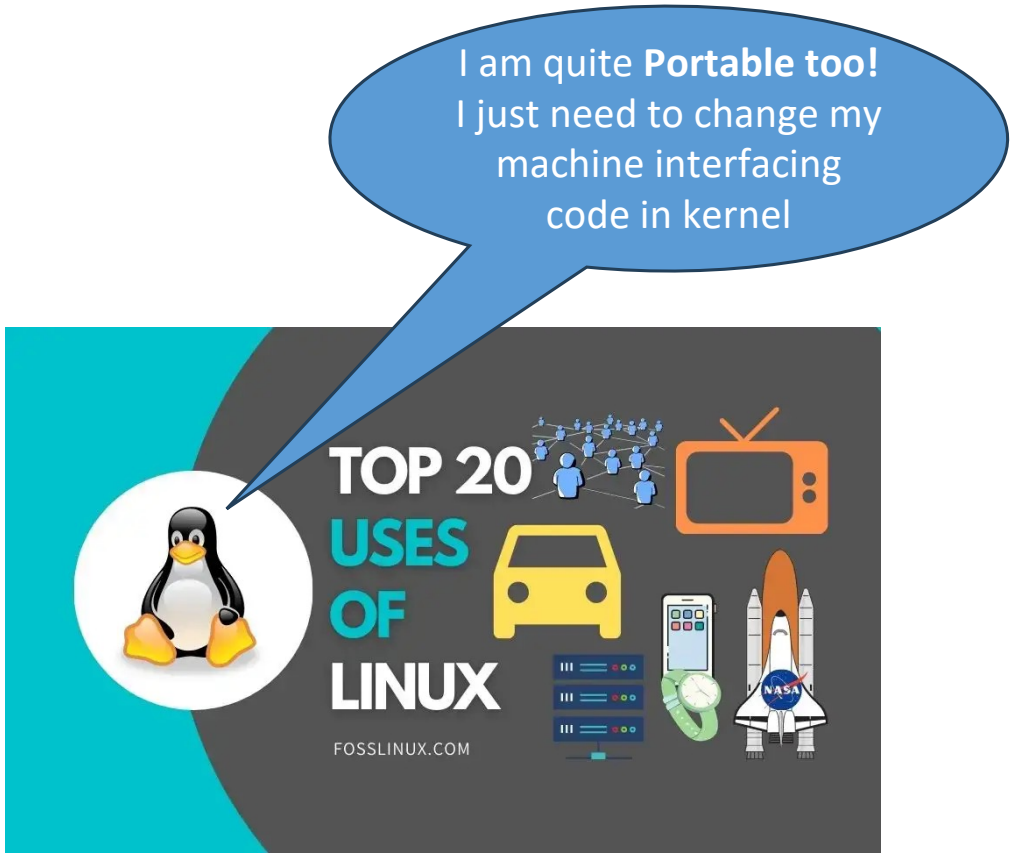
Each application can only access its own **baggage** (files, memory, CPU). They should not access another application's **baggage** unless they have proper authorization

Major Challenges in Modern OS

- Portability







Today's Lecture

- Modern computing architecture
- High-level overview of operating systems
- Roles of an OS
- Challenges in modern OS
- Course evaluation and logistics

Course Evaluation (Section-A)

- **Both sections will be graded SEPARATELY**
  - Evaluation components are same, but quizzes/assignments/exams will be **totally** different
  - Below mentioned N-1 policy specific to Section-A only
- Quiz: 10% (N-1 policy)
  - No surprise quizzes
  - Will be held during lecture hours (around 20mins duration)
- Semester exams
  - Mid-sem: 20%
  - End-sem: 35%
- Take home assignments 35%
  - **NO (N-1 policy) !!!**

I might have a different lecture plan/content than you see currently on Techtree. However, the COs would remain the same

Important Information

1. We will **Not** upload mid/end semester solution/rubric on Google Classroom
   - Although, we will discuss it in class
2. We will be taking in-class attendance
   - Lecture recordings will not be provided

Course Prerequisites

- **C programming and debugging skills are must!**
  - You must have used C in your DSA course and in OS refresher module
  - If you don't know C then better start practicing so as you don't face issues with the course assignments
    - First one will be released early next week!

We will strictly follow the IIITD **plagiarism policy**. No excuses if you get caught in plagiarism

You must declare explicitly in your assignments if your submission includes any AI-generated/internet code

Course Logistics

o4h2ev26

- Only **Section-A** students should register to above classroom
- Machines for assignments
  - You can use your laptop
    - Would require Linux OS (or any VM running Linux OS)
  - In case you have problems please feel free to first shoot an email to TF (Renu Singh), and then to me if your issues isn't resolved
- Course textbook is Operating Systems Three Easy Pieces (OSTEP)

Next Lecture

- Source code to machine code